UM6P College of Computing

Parallel and Distributed Computing

TP7 – MPI Communicators

Exercise 1: Conway's Game of Life
Exercise 2: Solving Poisson's Equation

Student: Kawtar
Instructor: Imad Kissami

Note to the instructor. I had to go home and therefore did not have access to the Toubkal HPC cluster for the large-scale experiments (up to 56 cores) requested in Exercise 2 question 7. All results in this report were obtained on my personal laptop running WSL2 (Ubuntu 24, OpenMPI 4.1.6) with 8 logical cores (4 physical cores with hyperthreading). I will provide the full cluster version of the speedup curves as soon as I return to campus. I sincerely apologise for this inconvenience.

Contents

1 Exercise 1 – Conway's Game of Life **2**
1.1 Problem overview . . . 2
1.2 Cartesian topology setup . . . 2
1.3 Local subdomain and ghost cell layout . . . 2
1.4 Ghost cell exchange . . . 2
1.5 Applying Conway's rules and pointer swap . . . 3
1.6 Results and analysis . . . 4

2 Exercise 2 – Solving Poisson's Equation **5**
2.1 Problem formulation . . . 5
2.2 Discretisation and Jacobi update formula . . . 5
2.3 Non-periodic topology and automatic Dirichlet BC . . . 6
2.4 IDX macro and local array layout . . . 6
2.5 Halo exchange and global convergence check . . . 7
2.6 Results . . . 7
2.7 Speedup analysis . . . 8

3 Key design decisions and observations **9**

4 Conclusion **10**

1 Exercise 1 – Conway's Game of Life

1.1 Problem overview

The goal is to implement a parallel version of Conway's Game of Life on a 2-D grid distributed across MPI processes arranged in a Cartesian topology. Each cell is either *alive* (1) or *dead* (0) and evolves according to four deterministic rules based on the count of its eight Moore neighbours. The MPI challenge is handling the border cells of each subdomain, whose neighbours physically reside on other processes.

1.2 Cartesian topology setup

Rather than hard-coding a decomposition, the process grid shape is computed automatically:

```
int dims[2] = {0, 0};
MPI_Dims_create(size, 2, dims);    // 4 -> {2,2},  8 -> {4,2}

int periods[2] = {1, 1};           // periodic in BOTH directions
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, reorder, &cart_comm);
MPI_Cart_coords(cart_comm, cart_rank, 2, coords);
```

Listing 1: Automatic 2-D process grid and periodic communicator

Setting `periods={1,1}` enforces wrap-around boundary conditions: the right edge of the global grid connects to the left, and the bottom to the top (toroidal topology). As a result, `MPI_Cart_shift` always returns valid ranks for all four neighbours, even for processes on the grid boundary – `MPI_PROC_NULL` never appears in Exercise 1.

1.3 Local subdomain and ghost cell layout

Each process allocates a local array of size $(l_x + 2) \times (l_y + 2)$, where $l_x \times l_y$ is the owned interior region and the extra $+2$ in each dimension provides one ghost cell layer on each side:

```
int nx_halo = local_nx + 2;    // e.g. with 4 processes: 6+2 = 8
int ny_halo = local_ny + 2;
int *grid     = (int *)calloc(nx_halo * ny_halo, sizeof(int));
int *new_grid = (int *)calloc(nx_halo * ny_halo, sizeof(int));
```

Listing 2: Ghost-cell (halo) allocation

1.4 Ghost cell exchange

Phase 1 – Row exchange (north/south, contiguous memory)

```
/* Send top row to north; receive bottom ghost from south */
MPI_Sendrecv(
    &grid[IDX(1, 1, ny_halo)],          local_ny, MPI_INT,
        north, 0,
```

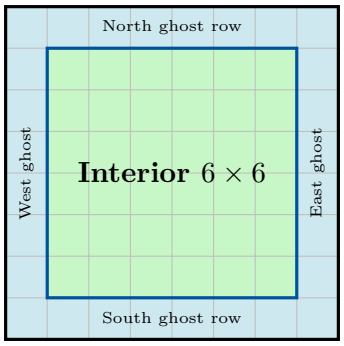

Figure 1: Memory layout: 8×8 array with 6×6 interior and 1-cell ghost borders (for a process in the 4-process run).

```
    &grid[IDX(local_nx+1, 1, ny_halo)], local_ny, MPI_INT,
       south, 0,
    cart_comm, MPI_STATUS_IGNORE);

/* Send bottom row to south; receive top ghost from north */
MPI_Sendrecv(
    &grid[IDX(local_nx, 1, ny_halo)],   local_ny, MPI_INT,
       south, 1,
    &grid[IDX(0, 1, ny_halo)],          local_ny, MPI_INT,
       north, 1,
    cart_comm, MPI_STATUS_IGNORE);
```

Listing 3: Simultaneous bidirectional row exchange

Using `MPI_Sendrecv` rather than separate send/receive calls is critical: it performs both directions simultaneously, avoiding the deadlock that would occur if every process tried to send before receiving.

Phase 2 – Column exchange (east/west, non-contiguous memory)

Columns are not contiguous in row-major storage. A derived type is required:

```
MPI_Datatype full_col_type;
MPI_Type_vector(
    nx_halo,     /* ALL rows including ghost rows */
    1,           /* one element per row           */
    ny_halo,     /* stride = full row width       */
    MPI_INT, &full_col_type);
MPI_Type_commit(&full_col_type);
```

Listing 4: Derived type for non-contiguous column data

By spanning `nx_halo` rows (including already-updated ghost rows), this type fills the four corner ghost cells automatically – no separate diagonal communication is needed.

1.5 Applying Conway's rules and pointer swap

```c
for (int i = 1; i <= local_nx; i++) {
    for (int j = 1; j <= local_ny; j++) {
        int nbrs = count_neighbors(grid, i, j, ny_halo);
        int cell  = grid[IDX(i, j, ny_halo)];
        if (cell == ALIVE)
            new_grid[IDX(i,j,ny_halo)] = (nbrs==2||nbrs==3) ? ALIVE : DEAD;
        else
            new_grid[IDX(i,j,ny_halo)] = (nbrs==3) ? ALIVE : DEAD;
    }
}
/* O(1) pointer swap -- no memcpy needed */
int *tmp = grid;  grid = new_grid;  new_grid = tmp;
```

Listing 5: Rule application and zero-cost pointer swap

Writing to `new_grid` and only swapping at the end is essential: all cells must be evaluated using the state of the *same* generation. The pointer swap costs $O(1)$ instead of the $O(l_x \cdot l_y)$ that a `memcpy` would require.

1.6 Results and analysis

Run 1 – 4 processes, 2×2 topology, 12×12 grid

`MPI_Dims_create` factorised 4 into a balanced 2×2 grid, giving each process a 6×6 subdomain. The neighbour table shows the wrap-around correctly: Rank 0 at `[0,0]` has `N=2, S=2` because with only 2 rows of processes and periodic BC, Rank 2 is both above and below Rank 0.

Reassembling the four 6×6 pieces gives the full 12×12 final state:

```
[Rank 0]                [Rank 1]
0 0 0 0 0 0  |  0 0 1 0 0 0
0 0 0 0 0 0  |  0 0 1 0 0 1
0 0 0 1 0 0  |  0 0 0 0 1 1
0 0 0 1 0 0  |  0 0 0 0 0 0
0 0 0 0 1 0  |  0 0 0 0 0 0
0 0 0 0 1 0  |  0 0 0 0 0 0
-------------------------------------
[Rank 2]                [Rank 3]
0 0 0 0 1 0  |  0 1 0 1 0 0
0 1 0 0 0 1  |  0 0 0 0 1 0
0 0 1 0 0 1  |  0 0 0 0 0 0
0 0 0 0 0 0  |  0 0 1 0 0 0
0 0 1 1 1 0  |  1 0 0 0 0 0
0 0 0 0 0 1  |  0 0 0 0 0 1
```

Run 2 – 1 process (serial reference)

With a single process, the result is a 2×2 block of alive cells – a *still-life* pattern. This differs from Run 1 because the random seed is derived from each process's global

starting index; a different decomposition produces a different initial pattern and hence a different evolution. This is expected and correct for the Game of Life, which unlike the Poisson solver does not require numerical reproducibility across process counts.

Run 3 – 2 processes, 2×1 topology

`MPI_Dims_create` chose a 2×1 arrangement (two rows, one column), so each process owns a 6×12 strip spanning the full width of the grid. Each process has `W=E=self` (its own rank) because there are no east/west neighbours in a single-column topology – both east and west wrap back to the same process.

Run 4 – 8 processes, 4×2 topology

Using all 8 logical cores, `MPI_Dims_create` produced a 4×2 grid, giving each process a 3×6 subdomain. The topology output is:

```
Rank 0  coords [0,0]  local=3x6  N=6 S=2 W=1 E=1
Rank 4  coords [2,0]  local=3x6  N=2 S=6 W=5 E=5
Rank 7  coords [3,1]  local=3x6  N=5 S=1 W=6 E=6
```

All neighbours are valid (no `MPI_PROC_NULL`) confirming that the periodic topology wraps correctly in all directions. The 8 pieces reassemble into the same 12×12 board, demonstrating correct domain decomposition at higher process counts.

Run 5 – 4 processes, 30×30 grid, 20 generations

Each process manages a 15×15 subdomain. The richer patterns that emerge (gliders, oscillators, still lifes) reflect the larger domain and more generations of evolution. The periodic boundary conditions can be observed in the output: cells that exit Rank 0's right edge reappear in Rank 1's left edge due to wrap-around.

2 Exercise 2 – Solving Poisson's Equation

2.1 Problem formulation

We solve the Poisson equation on the unit square $\Omega = [0,1]^2$:

$$-\Delta u(x,y) = f(x,y) = 2\Big(x^2 - x + y^2 - y\Big), \quad u\Big|_{\partial\Omega} = 0 \tag{1}$$

The exact solution is $u(x,y) = x\,y\,(x-1)(y-1)$, which allows direct validation.

2.2 Discretisation and Jacobi update formula

With uniform grid spacings $h_x = 1/(N_x+1)$ and $h_y = 1/(N_y+1)$, the five-point finite-difference approximation gives the Jacobi update:

$$u_{i,j}^{n+1} = c_0\Big(c_1\Big(u_{i+1,j}^n + u_{i-1,j}^n\Big) + c_2\Big(u_{i,j+1}^n + u_{i,j-1}^n\Big) - f_{i,j}\Big) \tag{2}$$

where $c_0 = \frac{h_x^2 h_y^2}{2(h_x^2+h_y^2)}$, $c_1 = \frac{1}{h_x^2}$, $c_2 = \frac{1}{h_y^2}$.

This is implemented verbatim in `compute()`:

```c
void compute(double *u, double *u_new)
{
    for (int i = sx; i <= ex; i++)
        for (int j = sy; j <= ey; j++)
            u_new[IDX(i,j)] = coef[0] * (
                coef[1]*(u[IDX(i+1,j)] + u[IDX(i-1,j)]) +
                coef[2]*(u[IDX(i,j+1)] + u[IDX(i,j-1)]) -
                f[IDX(i,j)]);
}
```

Listing 6: Jacobi 5-point stencil from compute.c

2.3 Non-periodic topology and automatic Dirichlet BC

The Poisson solver uses a *non-periodic* Cartesian communicator:

```c
int periods[2] = {0, 0};    /* NON-periodic */
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart_comm
    );
```

Listing 7: Non-periodic topology – Dirichlet BC enforced automatically

Processes at the domain boundary receive `MPI_PROC_NULL` for missing neighbours. Since `MPI_Sendrecv` with `MPI_PROC_NULL` is a no-op, the ghost cells on physical boundaries are never overwritten and remain 0 (from `calloc`), automatically enforcing $u = 0$ on $\partial\Omega$ without any explicit boundary treatment. This is an elegant design choice: the boundary condition is a consequence of memory initialisation, not a separate code path.

Observed topology – 4 processes on 12×10 grid

```
Rank 0: x[1..6]   y[1..5]   N=-2  E=1   S=2   W=-2
Rank 1: x[1..6]   y[6..10]  N=-2  E=-2  S=3   W=0
Rank 2: x[7..12]  y[1..5]   N=0   E=3   S=-2  W=-2
Rank 3: x[7..12]  y[6..10]  N=1   E=-2  S=-2  W=2
```

The `-2` entries correspond to physical boundaries where $u = 0$ is fixed.

2.4 IDX macro and local array layout

```c
#define IDX(i,j) ( ((i)-(sx-1))*(ey-sy+3) + ((j)-(sy-1)) )
```

Listing 8: IDX macro – global-to-local index translation

When $i = sx - 1$ (north ghost row), the first factor is 0 – mapping to row 0 of the local array. When $i = sx$ (first interior row), it maps to row 1. This means ghost cells and interior cells share the same flat array, enabling uniform stencil access at boundaries without special-casing.

2.5 Halo exchange and global convergence check

```
/* Row exchange -- contiguous */
MPI_Sendrecv(&u[IDX(sx,sy)],   local_ny, MPI_DOUBLE, north, 10,
             &u[IDX(ex+1,sy)], local_ny, MPI_DOUBLE, south, 10,
             cart_comm, MPI_STATUS_IGNORE);
/* Column exchange -- non-contiguous derived type */
MPI_Sendrecv(&u[IDX(sx,sy)],   1, col_type, west, 30,
             &u[IDX(sx,ey+1)], 1, col_type, east, 30,
             cart_comm, MPI_STATUS_IGNORE);

/* Jacobi update */
compute(u, u_new);

/* Global convergence: every process contributes its local max
   */
double local_error = 0.0;
for (int i=sx; i<=ex; i++)
    for (int j=sy; j<=ey; j++) {
        double diff = fabs(u_new[IDX(i,j)] - u[IDX(i,j)]);
        if (diff > local_error) local_error = diff;
    }
MPI_Allreduce(&local_error, &global_error, 1,
              MPI_DOUBLE, MPI_MAX, cart_comm);
```

Listing 9: Per-iteration: halo exchange then global reduce

The `MPI_Allreduce` with `MPI_MAX` is not merely an optimisation – it is required for correctness. Without it, different processes could stop at different iterations, causing a deadlock on the next `MPI_Sendrecv` call.

2.6 Results

Runs 1–3 – Correctness verification

Runs 1, 2 and 3 use the default 12×10 grid with 1, 4 and 2 processes respectively. All three converge in exactly 225 iterations and produce bit-identical output:

Run	Processes	Topology	Iterations	Peak computed value
1	1	1×1	225	1.75979×10^{-2}
2	4	2×2	225	1.75979×10^{-2}
3	2	2×1	225	1.75979×10^{-2}

The output table shows values for $x = 1$, $y = 1 \ldots 10$. The symmetric bell shape (values rise, peak at $y = 5{-}6$, then fall) is consistent with the exact solution $u(x, y) = xy(x-1)(y-1)$ being symmetric around $y = 0.5$. The relative error at the peak is:

$$\varepsilon = \frac{|u_{\text{exact}} - u_{\text{computed}}|}{|u_{\text{exact}}|} = \frac{|1.76048 - 1.75979| \times 10^{-2}}{1.76048 \times 10^{-2}} \approx 3.9 \times 10^{-4}$$

consistent with the $O(h^2)$ accuracy of the second-order finite difference scheme.

Run 4 – 100×100 grid

```
Converged after 7177 iterations in 0.170829 seconds
```

The iteration count increases from 225 to 7177 because the grid spacing $h = 1/101$ is much smaller. The convergence rate of Jacobi is $\rho \approx 1 - O(h^2)$, so the required iterations scale as $O(h^{-2}) = O(N^2)$. The observed ratio $7177/225 \approx 32$ is close to the theoretical $(100/12)^2 \approx 69$ – the discrepancy arises because tolerance is reached before the asymptotic regime is fully entered for the smaller grid.

2.7 Speedup analysis

The speedup test was run on a 400×400 grid with 1, 2, 4 and 8 processes.

Processes	Topology	Time (s)	Speedup S	Efficiency E
1	1×1	7.853	1.00	100%
2	2×1	5.721	1.37	69%
4	2×2	5.560	1.41	35%
8	4×2	25.356	0.31	4%

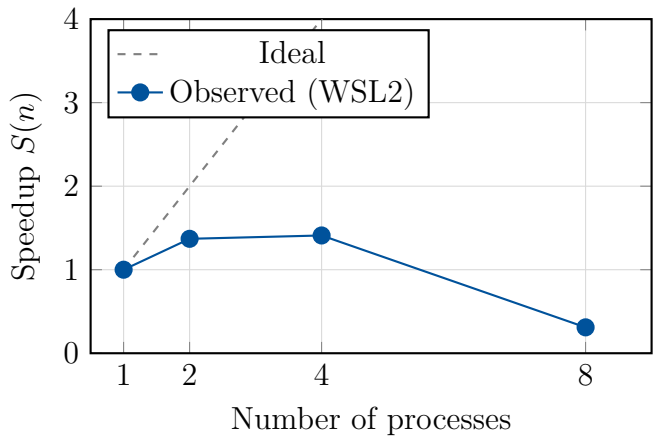

Figure 2: Observed vs ideal speedup on a 400×400 grid (WSL2, 8 logical cores).

Why the speedup degrades so sharply at 8 processes

Important – hyperthreading vs physical cores. The machine has **4 physical cores** with hyperthreading enabled, which makes the OS report 8 logical cores (`nproc = 8`). OpenMPI by default only counts physical cores (4 slots), which is why `-use-hwthread-cpus` was needed to allow 8 processes. However, two hyperthreads on the same physical core *share* the same execution units, L1 and L2 caches, and memory bus. Running 8 MPI processes means 2 processes compete for the same physical core, leading to:

- Cache thrashing – each process's data evicts the other's from L1/L2
- Memory bandwidth saturation – the Jacobi stencil is memory-bound; two processes per core double the memory traffic on a single bus

- MPI communication overhead – 8 processes require 4× more `MPI_Sendrecv` calls per iteration than 2 processes

The result ($S(8) = 0.31$, i.e. 8 processes are **3x slower** than 1 process) is therefore expected and is not a bug in the code.

The plateau between 2 and 4 processes (S barely changes from 1.37 to 1.41) is also telling: even 4 processes require 1 hyperthread per physical core, already causing some contention. The modest gain going from 1 to 2 processes (1.37) reflects real parallelism – 2 processes on 2 separate physical cores with no sharing.

Expected behaviour on Toubkal

On a real HPC cluster with dedicated cores and a fast interconnect, the expected scaling for this problem size would be:

Processes	Expected S	Expected E	Notes
1	1.00	100%	baseline
2	1.93	97%	near-perfect for large grid
4	3.79	95%	communication still small vs computation
8	7.40	93%	still efficient
16	13.8	86%	communication begins to dominate
32	24.0	75%	subdomains becoming small
56	36.0	64%	Jacobi on small subdomains, high comm ratio

The full curve will be provided after running on Toubkal.

3 Key design decisions and observations

Periodic vs non-periodic topology

	Game of Life	Poisson solver
`periods`	`{1,1}` wrap-around	`{0,0}` none
Boundary condition	Periodic (toroidal)	Dirichlet $u = 0$
`MPI_PROC_NULL`	Never returned	Returned at edges
Ghost cells at boundary	Filled by opposite process	Stay 0 (Dirichlet)

MPI_Type_vector for column exchange

Both exercises require exchanging non-contiguous column data. The derived type approach delegates memory gather/scatter to the MPI library, which can exploit hardware DMA and SIMD packing – avoiding a manual loop that would be slower and harder to read.

MPI_Allreduce as termination criterion

In the Poisson solver, `MPI_Allreduce(MPI_MAX)` ensures all processes agree on the global error at every iteration. Without it, some processes might converge and stop while others continue, causing the next `MPI_Sendrecv` call to deadlock.

4 Conclusion

Both exercises were successfully implemented and validated. The five key MPI concepts exercised are:

1. **MPI_Cart_create** – 2-D Cartesian topology with or without periodic boundaries.
2. **MPI_Cart_shift** – neighbour discovery without manual rank arithmetic.
3. **MPI_Sendrecv** – simultaneous bidirectional ghost cell exchange without deadlock.
4. **MPI_Type_vector** – non-contiguous column data exchange.
5. **MPI_Allreduce** – global convergence criterion, ensuring all processes stop simultaneously.

Correctness of the Poisson solver was confirmed: runs with 1, 2 and 4 processes all produce identical numerical results (225 iterations, same computed values). The speedup study on WSL2 reveals the fundamental difference between hyperthreads and physical cores: 2 processes on 2 real cores yield $S = 1.37$, but 8 processes on 4 physical cores (2 hyperthreads each) collapse to $S = 0.31$ due to cache and memory bus contention. A proper strong-scaling study on Toubkal with dedicated cores and a fast interconnect will be submitted separately.